Apuntes – Clase 17

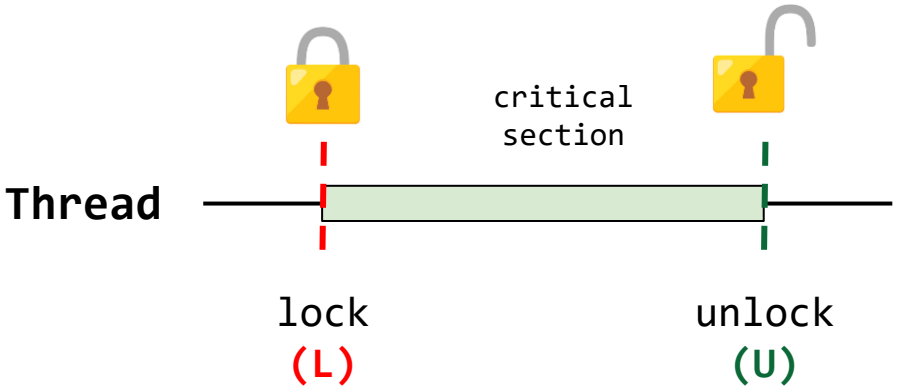
Variables de condición

Universidad de Antioquia

Facultad de Ingeniería

Ingeniería de Sistemas

Resumen clase anterior - Locks

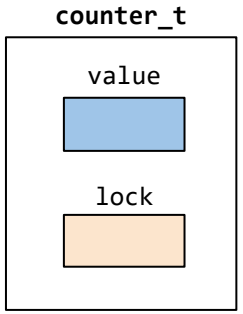



```
adquire(lock);
Región crítica
release(lock);
```

Resumen clase anterior – Estructuras de datos con Locks

Contador exacto

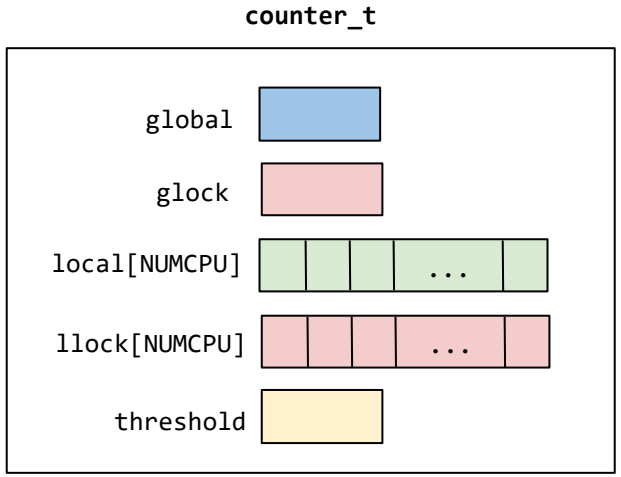
counter_t
value
lock




```
typedef struct __counter_t {
  int value;
  pthread_mutex_t lock;
} counter_t;
```

Contador aproximado

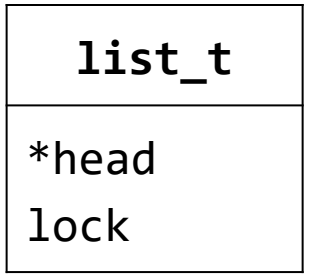
counter_t
global
glock
local[NUMCPU]
llock[NUMCPU]
threshold

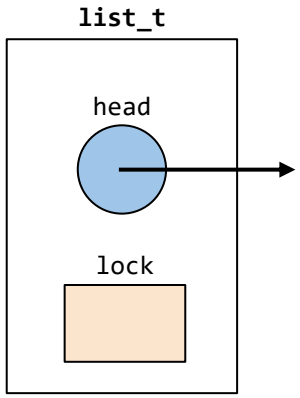



```
typedef struct __counter_t {
  int global;                       // global count
  pthread_mutex_t glock;            // global lock
  int local[NUMCPUS];               // local count (per cpu)
  pthread_mutex_t llock[NUMCPUS];   // ... and locks
  int threshold;                    // update frequency
} counter_t;
```

Resumen clase anterior – Estructuras de datos con Locks

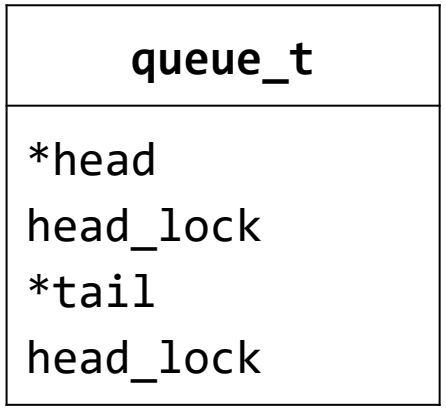
Lista enlazada

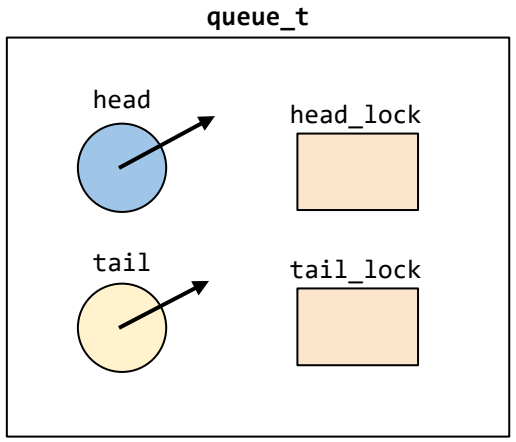





```
// basic list structure (one used per list)
typedef struct __list_t {
  node_t *head;
  pthread_mutex_t lock;
} list_t;
```

Cola

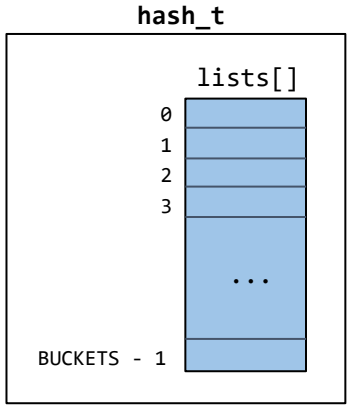





```
typedef struct __queue_t {
  node_t *head;
  node_t *tail;
  pthread_mutex_t head_lock, tail_lock;
} queue_t;
```

Resumen clase anterior – Estructuras de datos con Locks

Tabla hash




```
#define BUCKETS (101)

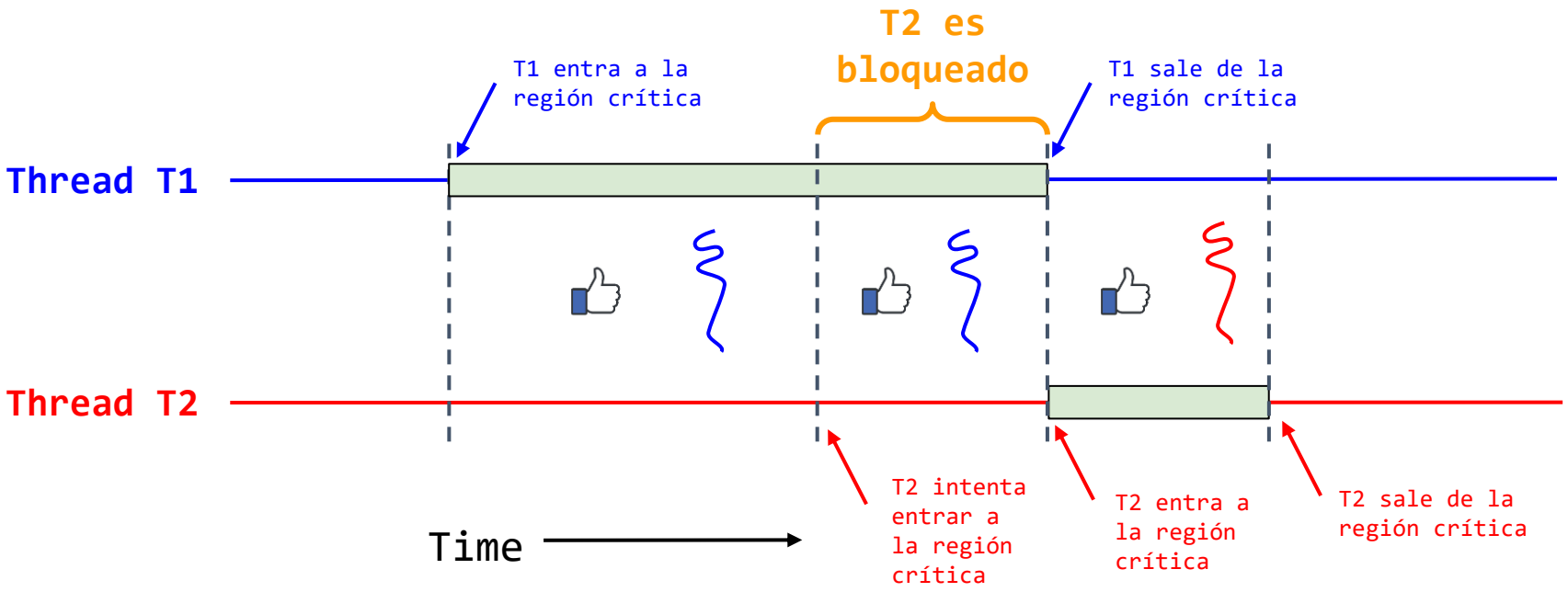
typedef struct __hash_t {
  list_t lists[BUCKETS];
} hash_t;
```



Contextualización

Objetivos de la concurrencia

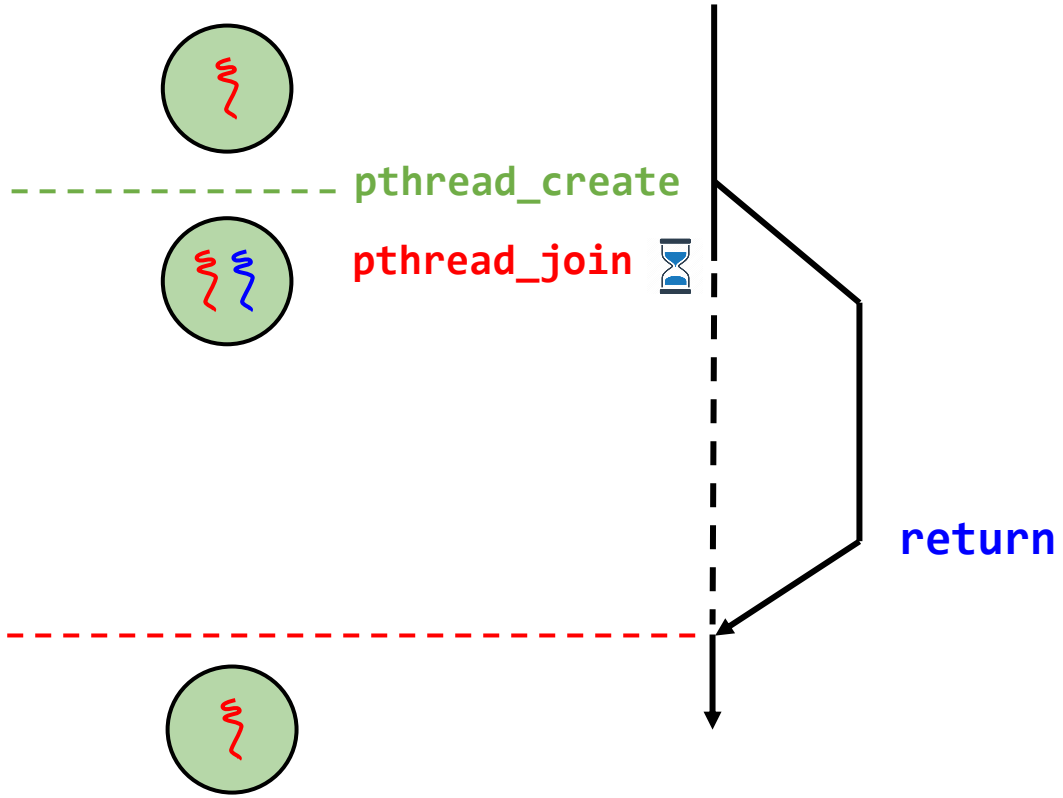
1. **Exclusión mutua**: que dos (o más) hilos (**T1** y **T2** por ejemplo) no se ejecuten a la misma vez) dentro de una misma sección crítica
   - **Solución**: **Locks**

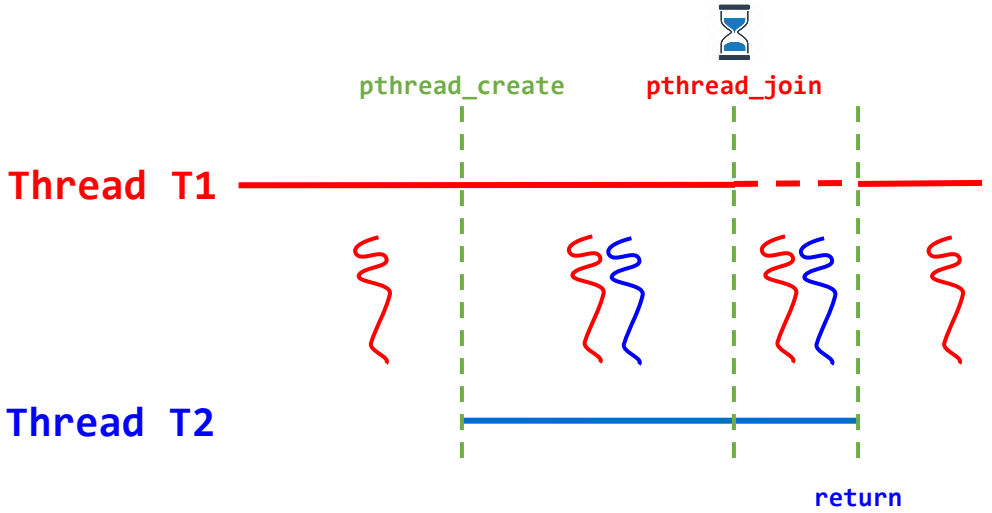


Contextualización

Objetivos de la concurrencia

2. **Orden en la ejecución (Ordering)**: Que un hilo (**T1** por ejemplo) corra después de que otro hilo haya hecho algo (**T2** por ejemplo)
   - **Solución**: **`Variables de condición (CV)`**.





Contextualización

¿Cómo lograr que se de un orden de ejecución entre los hilos?

- En ocasiones es necesario que un hilo **verifique que se cumpla una condición** antes de seguir avanzando.
  - **Ejemplo**:
    - El hilo de ejecución principal debe verificar que un hilo hijo ha finalizado.
    - Función join.

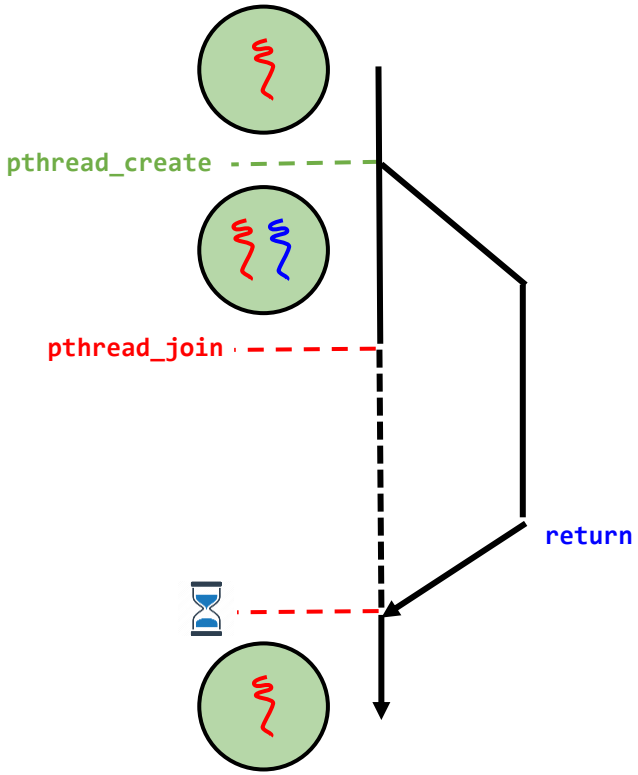




Contextualización

¿Cómo implementar el orden de ejecución deseado?

- **¿Cómo implementar el join?** (Para lograr el orden de ejecución deseado).




```
void *child(void *arg) {
  printf("child\n");
  // XXX how to indicate we are done?
  return NULL;
}

int main(int argc, char *argv[]) {
  printf("parent: begin\n");
  pthread_t c;
  Pthread_create(&c, NULL, child, NULL); //create
child
  // XXX how to wait for child?
  printf("parent: end\n");
  return 0;
}
```

Código

```
parent: begin
child
parent: end
```

Salida deseada

Contextualización

¿Cómo implementar el orden de ejecución deseado?

Spin join

- Poner al padre a esperar por el hijo.
- Se implementa usando una variable compartida (**done**)

```
parent: begin
child
parent: end
```

Salida deseada

```c
volatile int done = 0;

void *child(void *arg) {
  printf("child\n");
  done = 1;
  return NULL;
}

int main(int argc, char *argv[]) {
  printf("parent: begin\n");
  pthread_t c;
  pthread_create(&c, NULL, child, NULL); // create child
  while (done == 0)
          ; // spin
  printf("parent: end\n");
  return 0;
}
```

Código (link)

- **Solución ineficiente**: Desperdicio de CPU por parte del padre (debido al spin-loop).
- **Solución**: Hacer uso de variables de condición.

Contextualización

Variables de condición (Condition Variables - CV)

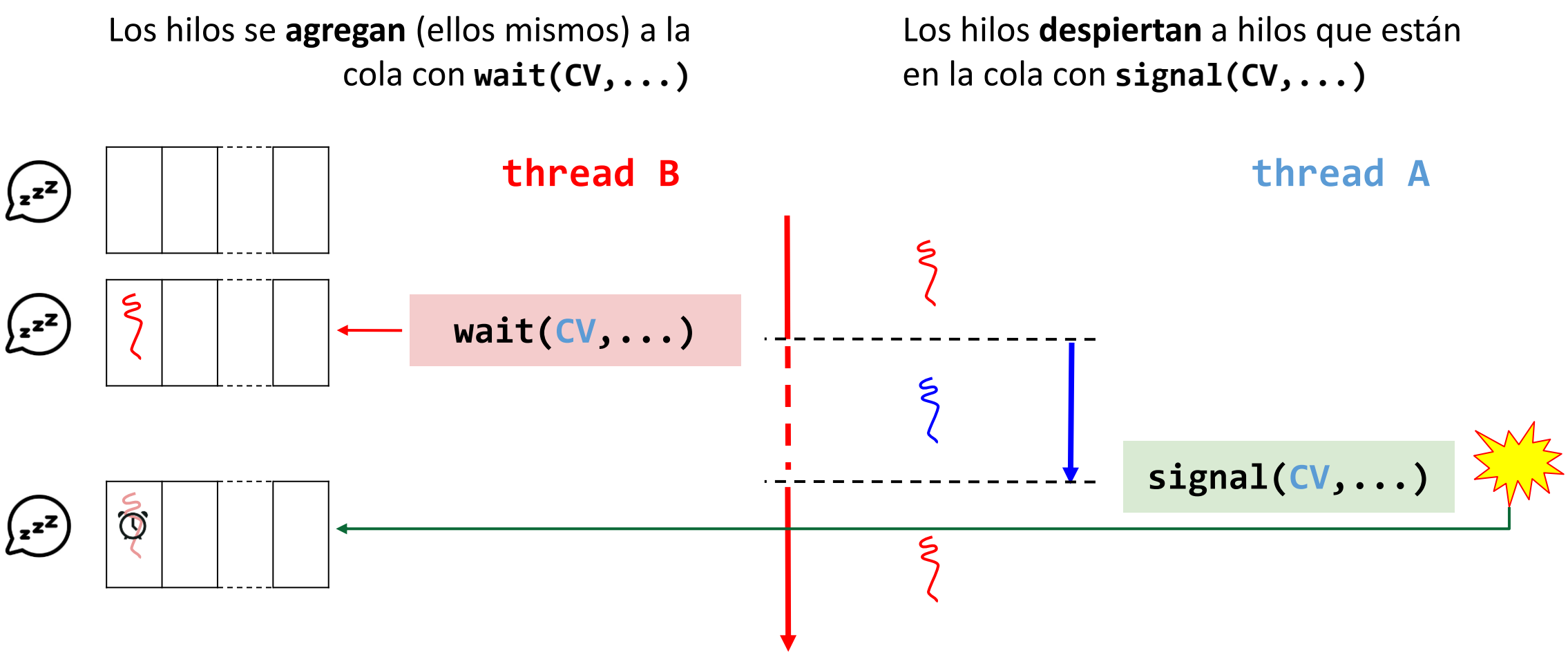
Idea clave: ¿Cómo esperar por una condición?

- **Espera de una condición:** En programas multihilo, es a menudo útil que un hilo espere hasta que alguna condición se vuelva verdadera antes de proseguir.
- **Solución ineficiente – Spinning:** Una solución es esperar validando continuamente hasta que la condición se vuelva verdadera (**spinning**) sin embargo esto es muy ineficiente debido al gasto de CPU.

¿Cómo implementar un hilo que espere por una condición?

Variables de condición

- Las **variables de condición (CV)** se usan para esperar hasta que alguna variable cumpla alguna condición (**evento**).
- **Variable de condición (CV):** Cola de hilos en espera.



Variables de condición

Operaciones de las variables de condición (CV)

Las variables de condición son un tipo de datos abstracto.

```
CV x, y;
```

Las únicas dos operaciones permitidas para una variable de condición son:

- **`x.wait()`**: El proceso que invoca la operación queda bloqueado hasta que otro proceso invoque **`x.signal()`**.
- **`x.signal()`**: Reanuda uno de los procesos bloqueados (cualquiera de los que invoco previamente el **`x.wait()`**).
  - Si no hay ningún proceso bloqueado, la operación **`signal`** no tiene efecto.

Variables de condición

Funciones Posix de las variables de condición

A continuación se describen las funciones Posix relacionadas con las operaciones en las variables de condición:

- **`wait(cond_t *cv, mutex_t *mutex)`**
  - Un hilo se pone en una cola (a dormir) a la espera de que un estado deseado suceda.
  - Recibe un mutex como parámetro.
  - Cuando se llama el **`wait`** el lock es liberado y se pone el hilo a dormir.
- **`signal(cond_t *cv)`**
  - Despierta un único hilo de los que se encuentran en la cola de espera (**`if >= 1`** el hilo está esperando).
  - Si no hay hilos esperando, solo retorna sin hacer nada.
  - Cuando el hilo se despierte se debe adquirir de nuevo el lock.

Variables de condición

Pthreads interface – Definición y funciones

Los **Mutexes** y las **CVs** son soportados en **Pthreads**

1. Declaración de una variable de condición (CV) y el **mutex**:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

Variables de condición

Pthreads interface – Definición y funciones

2. Operaciones de una variable de condición (CV):
   - **`wait()`**: Esta llamada es ejecutada cuando un hilo desea ponerse a sí mismo a dormir.

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
```

   - **`signal()`**: Esta llamada es realizada cuando un hilo ha cambiado algo en el programa y desea despertar un hilo en espera (que se encuentra durmiendo) cuando se da esta condición.

```
pthread_cond_signal(pthread_cond_t *c);
```

Implementación del join

Los Mutexes and **CVs** son soportados en Pthreads

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6    Pthread_mutex_lock(&m);
7    done = 1;
8    Pthread_cond_signal(&c);
9    Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13   printf("child\n");
14   thr_exit();
15   return NULL;
16 }
17
```

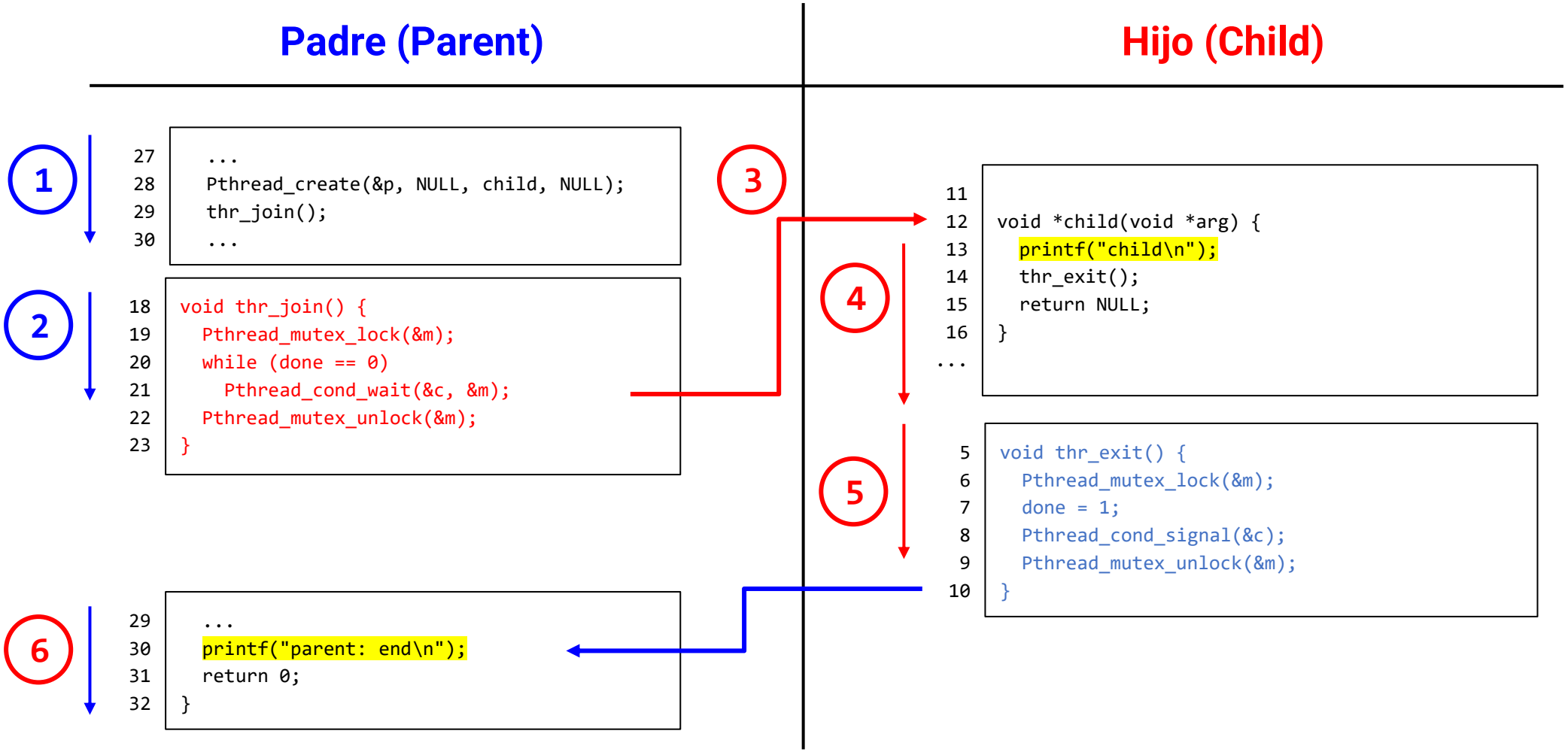
```
18 void thr_join() {
19   Pthread_mutex_lock(&m);
20   while (done == 0)
21         Pthread_cond_wait(&c, &m);
22   Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26   printf("parent: begin\n");
27   pthread_t p;
28   Pthread_create(&p, NULL, child, NULL);
29   thr_join();
30   printf("parent: end\n");
31   return 0;
32 }
```

Código (link)

Implementación del join

Análisis del join - Caso 1

El padre espera por el hijo; pero continúa corriendo así mismo de modo que llama inmediatamente a **`thr_join`**.



Implementación del join

Padre	Hijo	Anotaciones
`Pthread_create(&p, NULL, child, NULL);`		Crea el hilo hijo y continúa su ejecución.
`call thr_join();`		Llama función thr_join() para esperar finalización del hijo
`pthread_mutex_lock(&m);`		Adquiere el lock

`while (done == 0) {`
`  pthread_cond_wait(&c, &m);`
`}`
		Verifica si el hijo a finalizado. Para el caso como El hijo no ha finalizado (auno no ha tenido la oportunidad de ejecutarse). El padre Se pone en estado sleep llamando a la función wait.

	`call child()`	El hijo llama la función child
	`printf("child\n");`	Se imprime el mensaje: **child**
	`call thr_exit();`	Se llama la función thr_exit()
	`pthread_mutex_lock(&m);`	Obtiene el lock.
	`done = 1;`	Fija el valor de la variable done (done = 1)
	`pthread_cond_signal(&c);` `Pthread_mutex_unlock(&m);`	El hilo hijo señaliza al padre (despertandolo).

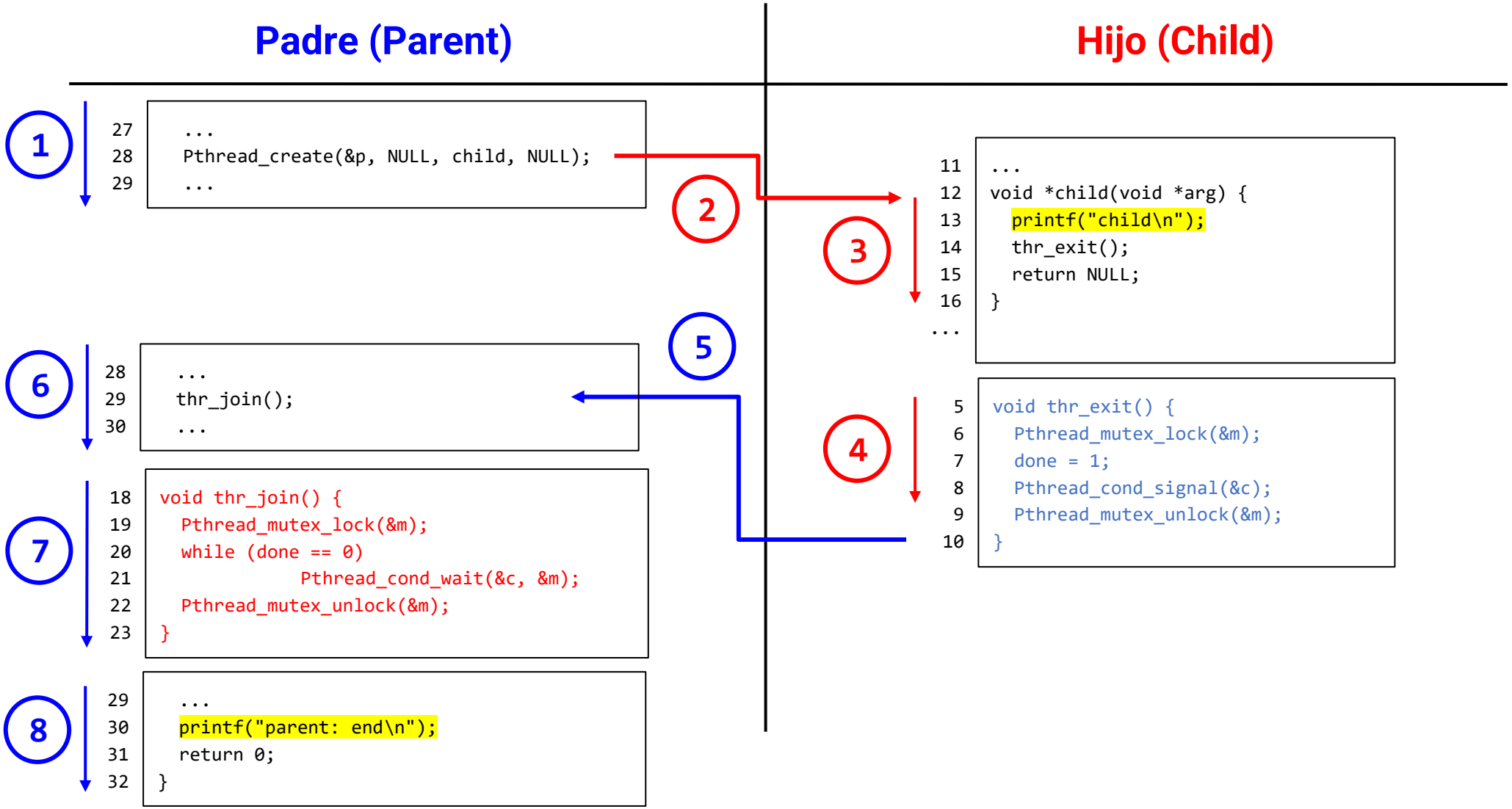
`while (done == 0) {`
`  pthread_cond_wait(&c, &m);`
`}`
		El padre retorna del wait (pues es despertado). En este caso ya done es 1.

`pthread_mutex_unlock(&m);`		Libera el lock.
`printf("parent: end\n");`		Imprime el mensaje **parent**

Implementación del join

Análisis del join - Caso 2

El hijo corre inmediatamente una vez es creado.



Implementación del join

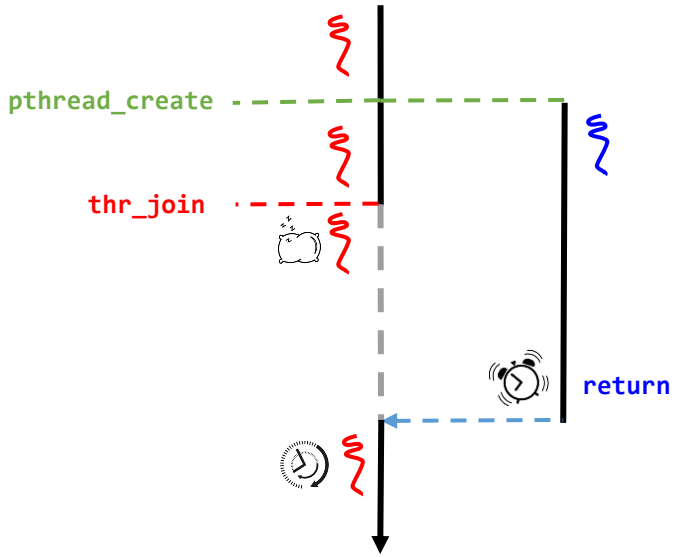
Padre	Hijo	Anotaciones
`Pthread_create(&p, NULL, child, NULL);`		Crea el hilo hijo
	`call child()`	El Hilo hijo empieza su ejecución llamando la función **child()**
	`printf("child\n");`	Se imprime el mensaje: **child**
	`call thr_exit();`	Llama función thr_exit() para despertar al hijo padre
	`pthread_mutex_lock(&m);`	Obtiene el lock.
	`done = 1;`	Se fija el valor de la variable done (done = 1)
	`pthread_cond_signal(&c);`	Llama signal para despertar al hilo padre. Como este no esta dormido, no hace nada, solo retorna.
	`Pthread_mutex_unlock(&m);`	Se libera el lock. Luego se pasa la ejecución al hilo padre.
`call thr_join();`		El hilo padre llama la función join
`pthread_mutex_lock(&m);`		El padre adquiere el lock. Tengase en cuenta que el lock se encuentra en 1 por lo que no se mete al ciclo que pone a dormir al hilo
`pthread_mutex_unlock(&m);`		Libera el lock.
`printf("parent: end\n");`		Imprime el mensaje **parent**

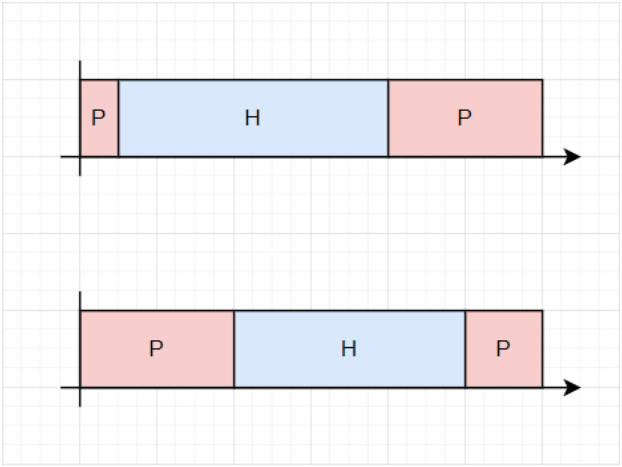
Implementación del join

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6    Pthread_mutex_lock(&m);
7    done = 1;
8    Pthread_cond_signal(&c);
9    Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13   printf("child\n");
14   thr_exit();
15   return NULL;
16 }
17
18 void thr_join() {
19   Pthread_mutex_lock(&m);
20   while (done == 0)
21     Pthread_cond_wait(&c, &m);
22   Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26   printf("parent: begin\n");
27   pthread_t p;
28   Pthread_create(&p, NULL, child, NULL);
29   thr_join();
30   printf("parent: end\n");
31   return 0;
32 }
```

Código (link)

Conclusión






```
parent: begin
child
parent: end
```

Reimplementación del join

¿Por qué es importante cada una de las instrucciones que se encuentran en **thr_join** y **thr_exit**?

Función thr_exit

```
void thr_exit() {
  Pthread_mutex_lock(&m);
  done = 1;
  Pthread_cond_signal(&c);
  Pthread_mutex_unlock(&m);
}
```

Función thr_join

```
void thr_join() {
  Pthread_mutex_lock(&m);
  while (done == 0)
        Pthread_cond_wait(&c, &m);
  Pthread_mutex_unlock(&m);
}
```

Intentemos reimplementar **thr_join** y thr_exit

Reimplementación del join

Reimplementación 1 (link)

Función `thr_exit`

```
void thr_exit() {
  Pthread_mutex_lock(&m);
  done = 1;
  Pthread_cond_signal(&c);
  Pthread_mutex_unlock(&m);
}
```

Función `thr_join`

```
void thr_join() {
  Pthread_mutex_lock(&m);
  while (done == 0)
        Pthread_cond_wait(&c, &m);
  Pthread_mutex_unlock(&m);
}
```



Función `thr_exit`

```
void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

Función `thr_join`

```
void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutex_unlock(&m);
}
```

Reimplementación del join

Reimplementación 1 (link)

Padre (Parent)

```
void thr_join() {
  pthread_mutex_lock(&m);      // x
  pthread_cond_wait(&c, &m);   // y
  pthread_mutex_unlock(&m);    // z
}
```

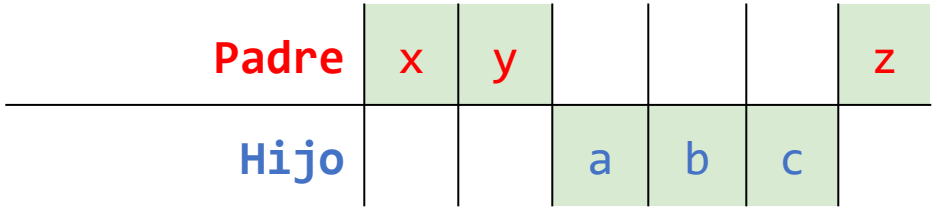
Hijo (Child)

```
void thr_exit() {
  pthread_mutex_lock(&m);    // a
  pthread_cond_signal(&c);   // b
  pthread_mutex_unlock(&m); // c
}
```

Análisis reimplementación 1

Caso 1: El padre crea un hilo hijo pero continúa corriendo así mismo y por lo tanto llama al join esperando a que el hilo hijo se complete. (**OK**).

```
pthread_create(&c, NULL, child, NULL);
thread_join();
```



Reimplementación del join

Reimplementación 1 (link)

Padre (Parent)

```
void thr_join() {
  pthread_mutex_lock(&m);      // x
  pthread_cond_wait(&c, &m);   // y
  pthread_mutex_unlock(&m);    // z
}
```

Hijo (Child)

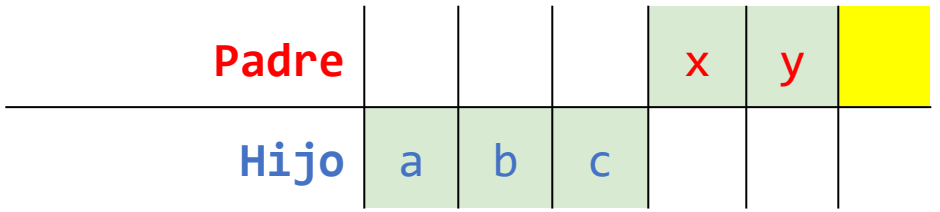
```
void thr_exit() {
  pthread_mutex_lock(&m);    // a
  pthread_cond_signal(&c);   // b
  pthread_mutex_unlock(&m); // c
}
```

Análisis reimplementación 1

Caso 2: El hilo hijo corre inmediatamente una vez es creado.

```
int main(int argc, char *argv[]) {
  ...
  Pthread_create(&p, NULL, child, NULL);
  thr_join();
  ...
  return 0;
}
```

```
void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
```



Problema: El padre esperará por siempre!!!

Reimplementación del join

Reimplementación 2 (link)

- **Regla de oro**: Además de usar CV es necesario **mantener el estado**.
- Los **CV** se utilizan para señalar hilos cuando cambia el estado.

Función `thr_exit`

```
void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}
```

Función `thr_join`

```
void thr_join() {
  pthread_mutex_lock(&m)
  if (done == 0) {
    pthread_cond_wait(&c);
  }
  pthread_mutex_unlock(&m);
}
```

Reimplementación del join

Reimplementación 2 (link)

Padre (Parent)

```
void thr_join() {
 pthread_mutex_lock(&m);       // w
 if (done == 0)                // x
   pthread_cond_wait(&c, &m); // y
 pthread_mutex_unlock(&m);     // z
}
```

Hijo (Child)

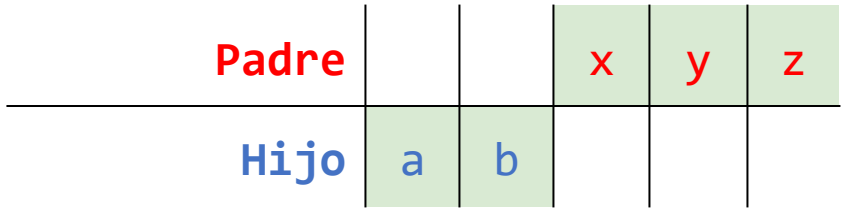
```
void thr_exit() {
  done = 1;                    // a
  pthread_cond_signal(&c);     // b
}
```

Análisis reimplementación 2

Caso 1: El hilo hijo corre inmediatamente una vez es creado (**Se arregla el problema anterior**).

```
int main(int argc, char *argv[]) {
  ...
  Pthread_create(&p, NULL, child, NULL);
  thr_join();
  ...
  return 0;
}
```

```
void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
```



Reimplementación del join

Reimplementación 2 (link)

Padre (Parent)

```
void thr_join() {
 pthread_mutex_lock(&m);       // w
 if (done == 0)                // x
   pthread_cond_wait(&c, &m); // y
 pthread_mutex_unlock(&m);     // z
}
```

Hijo (Child)

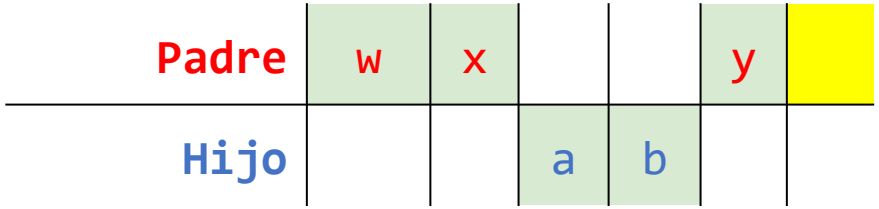
```
void thr_exit() {
  done = 1;                    // a
  pthread_cond_signal(&c);     // b
}
```

Análisis reimplementación 2

Caso 2: Una vez que el padre ha verificado el done es interrumpido antes de irse a dormir.

```
int main(int argc, char *argv[]) {
  ...
  Pthread_create(&p, NULL, child, NULL);
  thr_join();
  ...
  return 0;
}
```

```
void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
```



Problema: Race condition!!!

Reimplementación del join

Reimplementación del join 3 (link) - Regla de oro: El mutex es importante

- **Regla de oro**: Mantenga el **lock** siempre que llame el **signal** o el **wait**.
- Se usa el **mutex** para asegurarse que no se da una condición de competencia al interactuar con el estado (**done**) y con las llamadas **wait** y **signal**.

Función `thr_exit`

```
void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

Función `thr_join`

```
void thr_join() {
  pthread_mutex_lock(&m)
  while (done == 0) {
    pthread_cond_wait(&c, &m);
  }
  pthread_mutex_unlock(&m);
}
```

Reimplementación del join

Reimplementación del join 3 (link)

Padre (Parent)

```
void thr_join() {
 pthread_mutex_lock(&m);       // w
 while (done == 0)             // x
   pthread_cond_wait(&c, &m); // y
 pthread_mutex_unlock(&m);     // z
}
```

Hijo (Child)

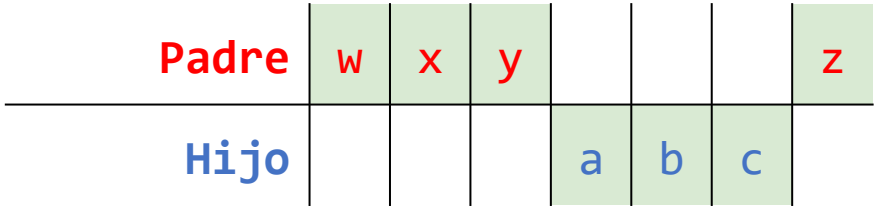
```
void thr_exit() {
  pthread_mutex_lock(&m);      // a
  done = 1;                    // b
  pthread_cond_signal(&c);     // c
  pthread_mutex_unlock(&m);    // d
}
```

Análisis reimplementación 3

Caso 1: Una vez que el padre ha verificado el done es interrumpido antes de irse a dormir **(Se soluciona el problema de la race condition)**.

```
int main(int argc, char *argv[]) {
  ...
  Pthread_create(&p, NULL, child, NULL);
  thr_join();
  ...
  return 0;
}
```

```
void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
```



Resumen implementación del join

Hemos visto los tres casos de implementación del join

- **Caso 1**: **Falla** cuando el hilo hijo corre inmediatamente una vez es creado.
- **Caso 2**: **Falla** cuando el padre es interrumpido inmediatamente después de que se da la validación del **done** antes de ponerse a dormir.
- **Caso 3**: Los problemas de las 2 implementaciones anteriores ya es **solucionado**.

Implementación – caso 1

Implementación del join - Caso 1: Falla cuando el hilo hijo corre inmediatamente una vez es creado.

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}

int main(int argc, char *argv[]) {
  printf("parent: begin\n");
  pthread_t p;
  pthread_create(&p, NULL, child, NULL);
  thr_join();
  printf("parent: end\n");
  return 0;
}
```

```
void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

```
void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutex_unlock(&m);
}
```

Implementación – Caso 2

Implementación del join - Caso 2: Falla cuando el padre es interrumpido inmediatamente después de que se da la validación del **done** antes de ponerse a dormir.

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
int main(int argc, char *argv[]) {
  printf("parent: begin\n");
  pthread_t p;
  pthread_create(&p, NULL, child, NULL);
  thr_join();
  printf("parent: end\n");
  return 0;
}
```

```
void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}
```

```
void thr_join() {
  pthread_mutex_lock(&m);
  if (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutex_unlock(&m);
}
```

Implementación – Caso 3

Implementación del join - Caso 3: Los problemas de las 2 implementaciones anteriores ya es solucionado

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void *child(void *arg) {
  printf("child\n");
  thr_exit();
  return NULL;
}
int main(int argc, char *argv[]) {
  printf("parent: begin\n");
  pthread_t p;
  pthread_create(&p, NULL, child, NULL);
  thr_join();
  printf("parent: end\n");
  return 0;
}
```

```
void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

```
void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutex_unlock(&m);
}
```

Problema del productor consumidor

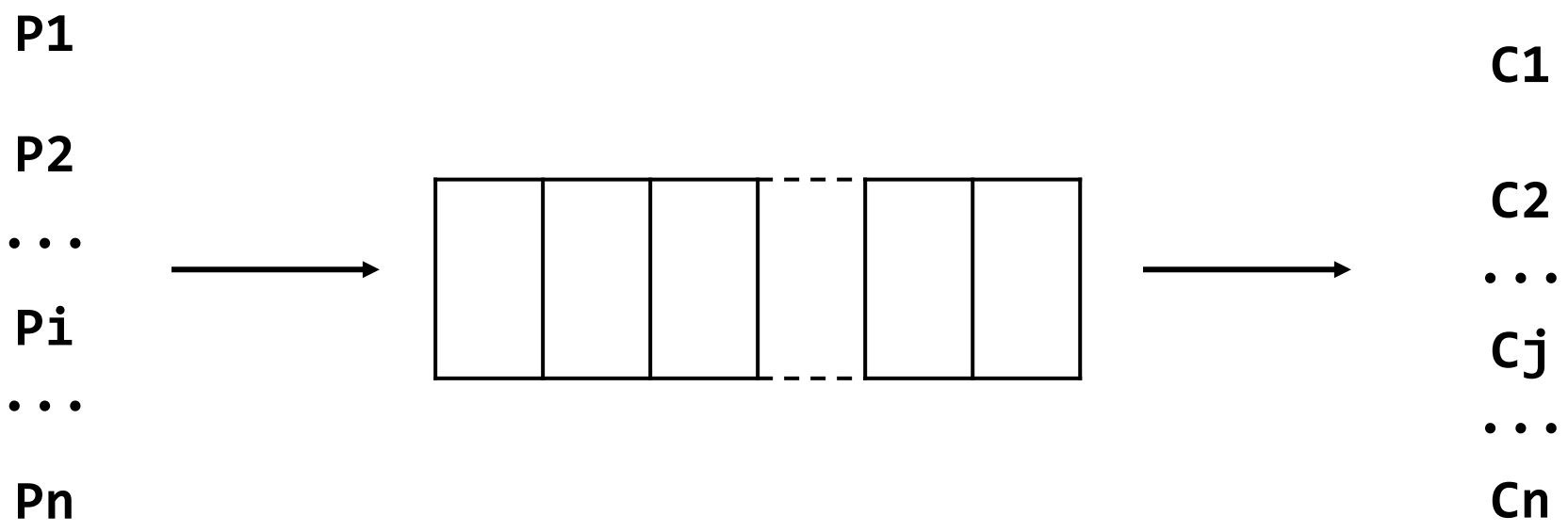
Problema del productor consumidor (Consumer/producer problem) también conocido como **problema del buffer limitado (bounded buffer problem)**

Productor

- Produce ítems (datos).
- Ubica los datos en un buffer.

Consumidor

- Retira datos del buffer y los consume.



Problema del productor consumidor

Ejemplos productor consumidor

Vida real



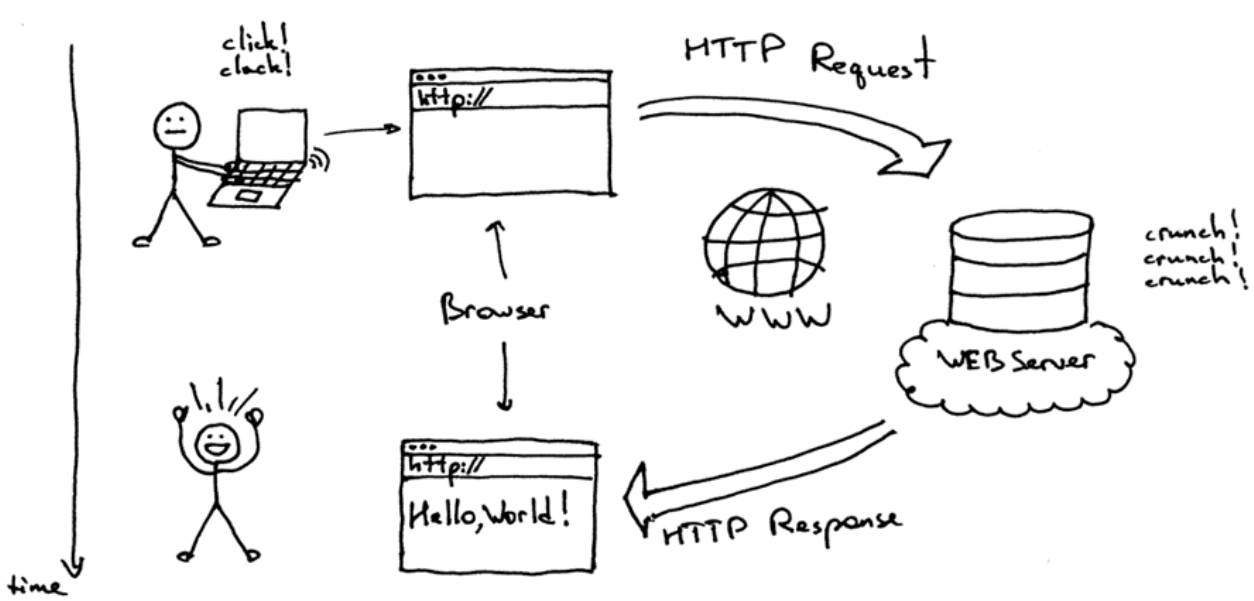


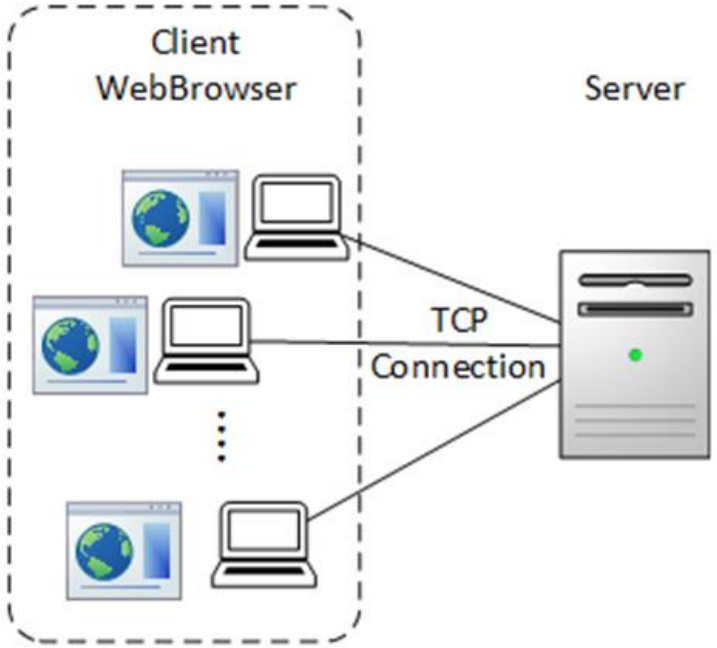
Problema del productor consumidor

Ejemplos productor consumidor

Servidor multi-hilo

- Un **productor** ingresa **http request** en una **cola** de trabajo.
- El **consumidor** retira requerimientos de la **cola** y los procesa.





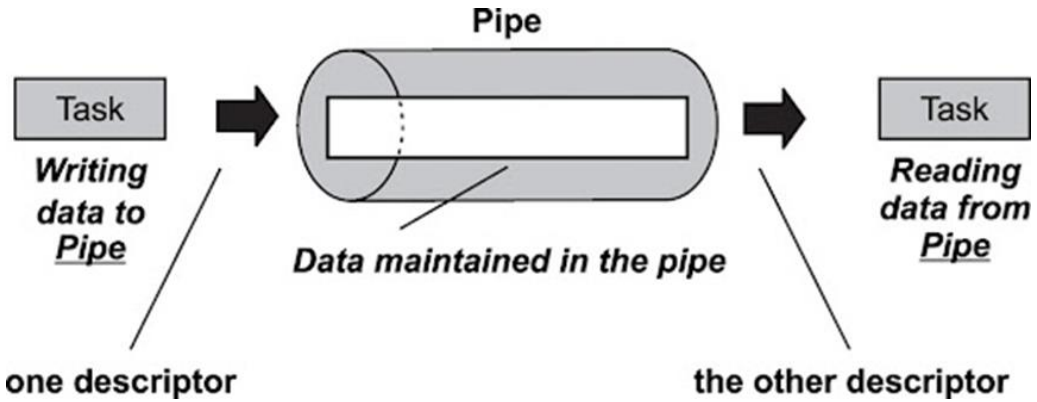
Problema del productor consumidor

Ejemplos productor consumidor

Tuberia

- Un bounded buffer (buffer limitado) es usado para llevar la salida de un programa como entrada de otro.
- **Ejemplo**: `grep foo file.txt | wc`

Productor: `grep`



Consumidor: `wc`

Buffer limitado: Tuberia

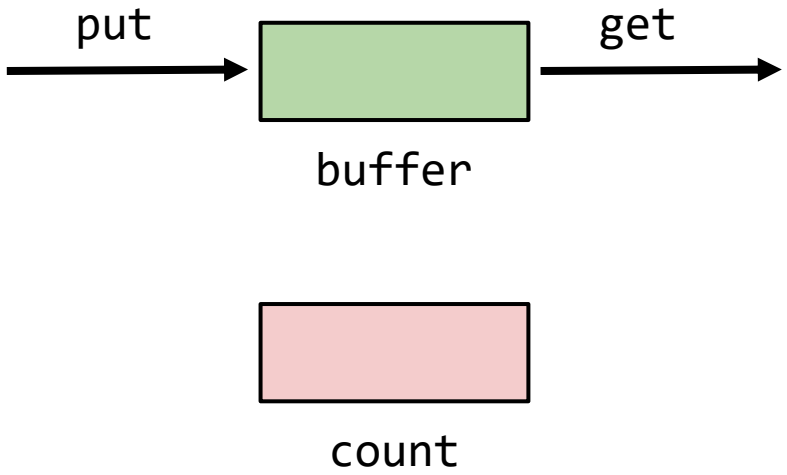
- **El buffer (bounded buffer) es el recurso compartido** → Requiere un acceso sincronizado.

Rutinas put y get

```
int buffer;
int count = 0; // initially, empty

void put(int value) {
  assert(count == 0);
  count = 1;
  buffer = value;
}

int get() {
  assert(count == 1);
  count = 0;
  return buffer;
}
```



- Solo ingrese datos al buffer cuando esté vació (**count == 0**).
- Solo retire datos del buffer está lleno (**count == 1**).

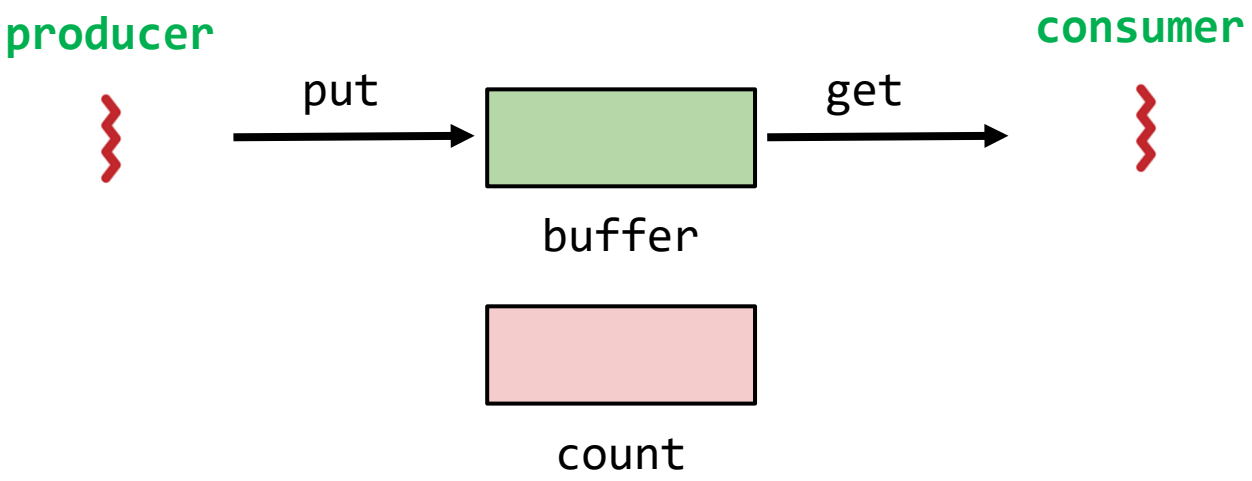
Hilos productor/consumidor

```
void *producer(void *arg) {
  int i;
  int loops = (int) arg;
  for (i = 0; i < loops; i++) {
    put(i);
  }
}

void *consumer(void *arg) {
  while (1) {
    int tmp = get();
    printf("%d\n", tmp);
  }
}
```

producer Ingresa datos enteros en el buffer compartido (uno por cada ciclo).

consumer Retira datos del buffer compartido.



- Solo ingrese datos al buffer cuando esté vació (**count == 0**).
- Solo retire datos del buffer está lleno (**count == 1**).

Productor/consumidor: Unica CV y sentencia if

Estrategia general:

Use **variables de condición** (CV) para hacer que los **consumidores esperen** cuando no haya nada que consumir (buffer vacío) y los **productores esperen** cuando el buffer se encuentre lleno.

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    if (count == 1)                      // p2
      pthread_cond_wait(&cond, &mutex);  // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while (1) {
    pthread_mutex_lock(&mutex);          // c1
    if (count == 0)                      // c2
      pthread_cond_wait(&cond, &mutex);  // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

- Se usa una única variable de condición (**cond**) y un lock (**mutex**) asociado.

Productor/consumidor: Unica CV y sentencia if

Lógica de señalización entre el productor y el consumidor:

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);            // p1
    if (count == 1)                        // p2
      pthread_cond_wait(&cond, &mutex);    // p3
    put(i);                                // p4
    pthread_cond_signal(&cond);            // p5
    pthread_mutex_unlock(&mutex);          // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while (1) {
    pthread_mutex_lock(&mutex);            // c1
    if (count == 0)                        // c2
      pthread_cond_wait(&cond, &mutex);    // c3
    tmp = get();                           // c4
    pthread_cond_signal(&cond);            // c5
    pthread_mutex_unlock(&mutex);          // c6
    printf("%d\n", tmp);
  }
}
```

producer

p1 - p3:
El productor espera a que el ***buffer*** esté vacío.

consumer

c1 - c3:
El Consumidor espera a que el ***buffer*** esté lleno.

Productor/consumidor: Unica CV y sentencia if

Análisis de la implementación

Caso	Escenario	Conclusiones
1	Un solo productor y un solo consumidor	El código funciona.
2	Un solo productor y varios consumidores	La solución tiene 2 problemas críticos (En breve veremos).

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);            // p1
    if (count == 1)                        // p2
      pthread_cond_wait(&cond, &mutex);    // p3
    put(i);                                // p4
    pthread_cond_signal(&cond);            // p5
    pthread_mutex_unlock(&mutex);          // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while (1) {
    pthread_mutex_lock(&mutex);            // c1
    if (count == 0)                        // c2
      pthread_cond_wait(&cond, &mutex);    // c3
    tmp = get();                           // c4
    pthread_cond_signal(&cond);            // c5
    pthread_mutex_unlock(&mutex);          // c6
    printf("%d\n", tmp);
  }
}
```

Productor/consumidor: Unica CV y sentencia if

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    if (count == 1)                      // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while (1) {
    pthread_mutex_lock(&mutex);          // c1
    if (count == 0)                      // c2
      pthread_cond_wait(&cond, &mutex);  // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

Caso 2 - Un solo productor y varios consumidores



T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Running	0	
	Sleep		Ready	p2	Running	0	
	Sleep		Ready	p4	Running	1	Buffer now full
	Ready		Ready	p5	Running	1	T_{c1} awoken
	Ready		Ready	p6	Running	1	
	Ready		Ready	p1	Running	1	
	Ready		Ready	p2	Running	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep
	Ready	c1	Running		Sleep	1	T_{c2} sneaks in ...
	Ready	c2	Running		Sleep	1	
	Ready	c4	Running		Sleep	0	... and grabs data
	Ready	c5	Running		Ready	0	T_p awoken
	Ready	c6	Running		Ready	0	
c4	Running		Ready		Ready	0	Oh oh! No data

Productor/consumidor: Unica CV y sentencia if

Caso 2 - Un solo productor y varios consumidores

Problema 1 – No hay garantía de que cuando un hilo despertado empiece su ejecución, el estado seguirá siendo el deseado: Después de que el productor (**Tp**) despierta a **Tc1**, pero antes de que **Tc1** se ejecute, el estado del buffer es cambiado por **Tc2**.

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Running	0	
	Sleep		Ready	p2	Running	0	
	Sleep		Ready	p4	Running	1	Buffer now full
	Ready		Ready	p5	Running	1	T_{c1} awoken
	Ready		Ready	p6	Running	1	
	Ready		Ready	p1	Running	1	
	Ready		Ready	p2	Running	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep
	Ready	c1	Running		Sleep	1	T_{c2} sneaks in ...
	Ready	c2	Running		Sleep	1	
	Ready	c4	Running		Sleep	0	... and grabs data
	Ready	c5	Running		Ready	0	T_p awoken
	Ready	c6	Running		Ready	0	
c4	Running		Ready		Ready	0	Oh oh! No data

Productor				p1	p2	p4	p5	p6	p1	p2	p3					
Consumidor 1	c1	c2	c3													c4
Consumidor 2												c1	c2	c4	c5	c6

Productor/consumidor: Unica CV y sentencia if

Código

Archivos:

- mythreads.h
- main_singleBuffer-v1.c

Compilación:

```
gcc -I. main_singleBuffer-v1.c -Wall -lpthread -o single_buffer-if1.out
```

Ejecución:

```
./single_buffer-if1.out <loops> <consumers>
```

Productor/consumidor: Unica CV y sentencia if

Pruebas

Caso	Escenario	Conclusiones
1	Un solo productor y un solo consumidor	El código funciona.

```
$ ./single_buffer-if1.out 10 1
0
1
2
3
4
5
6
7
8
9
^C
```

```
$ ./single_buffer-if1.out 20 1
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
^C
```

Productor/consumidor: Unica CV y sentencia if

Pruebas

Caso	Escenario	Conclusiones
2	Un solo productor y varios consumidores	El resultado no es el esperado

```
$ ./single_buffer-if1.out 10 2
0
2
1
3
5
6
4
7
9
8
^C
```

```
$ ./single_buffer-if1.out 30 2
0
2
1
3
4
5
7
8
6
10
9
11
13
14
15
16
12
18
17
19
single_buffer-if1.out: main_singleBuffer-v1.c:72: get: Assertion `count == 1' failed.
Aborted (core dumped)
```

Productor/consumidor: Unica CV y sentencia while

Estrategia general:

Cambie la sentencia **`if`** por **`while`** en el consumidor y en el productor

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    while (count == 1)                   // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
    pthread_mutex_lock(&mutex);          // c1
    while (count == 0)                   // c2
      pthread_cond_wait(&cond, &mutex);  // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

- Si el consumidor **Tc1** despierta y verifica nuevamente el estado de la variable.
  - Si el buffer está vacío, el consumidor vuelve a dormir.

Productor/consumidor: Unica CV y sentencia while

Estrategia general:

- La idea es que el hilo en espera siempre debe volver a verificar la condición (**cond**) en un **while**, en lugar de hacerlo en un **if**.
  - Si no se vuelve a verificar, el hilo que está en espera seguirá pensando que la condición ha cambiado a pesar de que no ha hecho.

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    while (count == 1)                   // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
    pthread_mutex_lock(&mutex);          // c1
    while (count == 0)                   // c2
      pthread_cond_wait(&cond, &mutex);  // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

Productor/consumidor: Unica CV y sentencia while

Estrategia general:

- **Regla de oro**: Con variables de condición (**CV**), se debe **usar siempre ciclos while**.
  - Todavía persiste el **problema 2**

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    while (count == 1)                   // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
    pthread_mutex_lock(&mutex);          // c1
    while (count == 0)                   // c2
      pthread_cond_wait(&cond, &mutex);  // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

Productor/consumidor: Unica CV y sentencia while

```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    while (count == 1)                   // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // c1
    while (count == 0)                   // c2
      pthread_cond_wait(&cond, &mutex); // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```



Análisis del problema: Problema 2

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Running		Ready	0	
	Sleep	c2	Running		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Running	0	
	Sleep		Sleep	p2	Running	0	
	Sleep		Sleep	p4	Running	1	Buffer now full
	Ready		Sleep	p5	Running	1	T_{c1} awoken
	Ready		Sleep	p6	Running	1	
	Ready		Sleep	p1	Running	1	
	Ready		Sleep	p2	Running	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Running		Sleep		Sleep	1	Recheck condition
c4	Running		Sleep		Sleep	0	T_{c1} grabs data
c5	Running		Ready		Sleep	0	Oops! Woke T_{c2}

Productor/consumidor: Unica CV y sentencia while

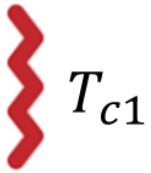
```
int loops; // must initialize somewhere...
cond_t cond;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // p1
    while (count == 1)                   // p2
      pthread_cond_wait(&cond, &mutex); // p3
    put(i);                              // p4
    pthread_cond_signal(&cond);          // p5
    pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);          // c1
    while (count == 0)                   // c2
      pthread_cond_wait(&cond, &mutex); // c3
    tmp = get();                         // c4
    pthread_cond_signal(&cond);          // c5
    pthread_mutex_unlock(&mutex);        // c6
    printf("%d\n", tmp);
  }
}
```

Análisis del problema: Problema 2



T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
...	...	...	...	...	...	...	*(cont.)*
c6	Running		Ready		Sleep	0	
c1	Running		Ready		Sleep	0	
c2	Running		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Running		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep ...





Conclusión:

1. Señalizar es necesario.
2. La señalización llevada a cabo debe ser **más específica**.

Productor/consumidor: Unica CV y sentencia while

Análisis del problema:

- Dos consumidores se ejecutan primero (**Tc1** y **Tc2**) y luego ambos van a dormir (**c3**).
- El productor (**Tp**) se despierta, pone un valor en el buffer y despierta a uno de los consumidores (**Tc1** por ejemplo).
- El productor (**Tp**) nuevamente se ejecuta e intenta poner mas datos en el buffer pero como el buffer esta lleno, el productor se va a dormir.
- Un consumidor esta listo para correr (**Tc1**) y dos hilos están durmiendo (**Tc2** y **Tp**).
- El consumidor (**Tc1**) despierta retornando del wait (**c2**), encuentra el buffer lleno y consume el valor (**c4**) vaciando el buffer.
- El consumidor señaliza (**c5**) despertando solo a uno de los dos hilos que se encuentra dormido (Supongamos para infortunio que es a **Tc2**)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Running		Ready	0	
	Sleep	c2	Running		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Running	0	
	Sleep		Sleep	p2	Running	0	
	Sleep		Sleep	p4	Running	1	Buffer now full
	Ready		Sleep	p5	Running	1	T_{c1} awoken
	Ready		Sleep	p6	Running	1	
	Ready		Sleep	p1	Running	1	
	Ready		Sleep	p2	Running	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Running		Sleep		Sleep	1	Recheck condition
c4	Running		Sleep		Sleep	0	T_{c1} grabs data
c5	Running		Ready		Sleep	0	Oops! Woke T_{c2}

Productor							p1	p2	p4	p5	p6	p1	p2	p3							
Consumidor 1	c1	c2	c3												c2	c4	**c5**	c6	c1	c2	c3		
Consumidor 2				c1	c2	c3																c2	c3

Productor/consumidor: Unica CV y sentencia while

Análisis del problema:

- Al despertarse (**Tc2**) y chequear el buffer (**c2**), lo encuentra vacío, por lo que se vuelve a dormir (**c3**).
- Con esto ya todos se encuentran dormidos…

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
...	...	...	...	...	...	...	*(cont.)*
c6	Running		Ready		Sleep	0	
c1	Running		Ready		Sleep	0	
c2	Running		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Running		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep ...

Conclusiones:

- Es necesario saber a quien despertar.
- Un consumidor sólo debe despertar productores (no a otros consumidores) y viceversa.

Productor							p1	p2	p4	p5	p6	p1	p2	p3								
Consumidor 1	c1	c2	c3												c2	c4	c5	c6	c1	c2	c3	
Consumidor 2				c1	c2	c3															c2	c3

Productor/consumidor: Unica CV y sentencia while

Código

Archivos:

- mythreads.h
- main_singleBuffer-v2.c

Compilación:

```
gcc -I. main_singleBuffer-v2.c -Wall -lpthread -o single_buffer-while1.out
```

Ejecución:

```
./single_buffer-while1.out <loops> <consumers>
```

Productor/consumidor: Unica CV y sentencia while

Pruebas

Escenario	Conclusiones
Un solo productor y varios consumidores	El código se puede bloquear

```
$ ./single_buffer-while1.out 1000 2
```

```
80
81
82
84
83
86
87
88
85
90
89
92
91
93
```

Productor/consumidor – Single buffer: Usar 2 variables de condición y un while.

Solución: Use dos variables de condición (`empty` y `fill`)

- **Productor**: va a dormir usando la variable `empty`, y señaliza `fill`
- **Consumidor**: va a dormir usando la variable `fill`, y señaliza `empty`

```
cond_t empty, fill;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
          pthread_mutex_lock(&mutex);
          while (count == 1)
            pthread_cond_wait(&empty, &mutex);
          put(i);
          pthread_cond_signal(&fill);
          pthread_mutex_unlock(&mutex);
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
          pthread_mutex_lock(&mutex);
          while (count == 0)
            pthread_cond_wait(&fill, &mutex);
          int tmp = get();
          pthread_cond_signal(&empty);
          pthread_mutex_unlock(&mutex);
          printf("%d\n", tmp);
  }
}
```

Productor/consumidor – Solución final

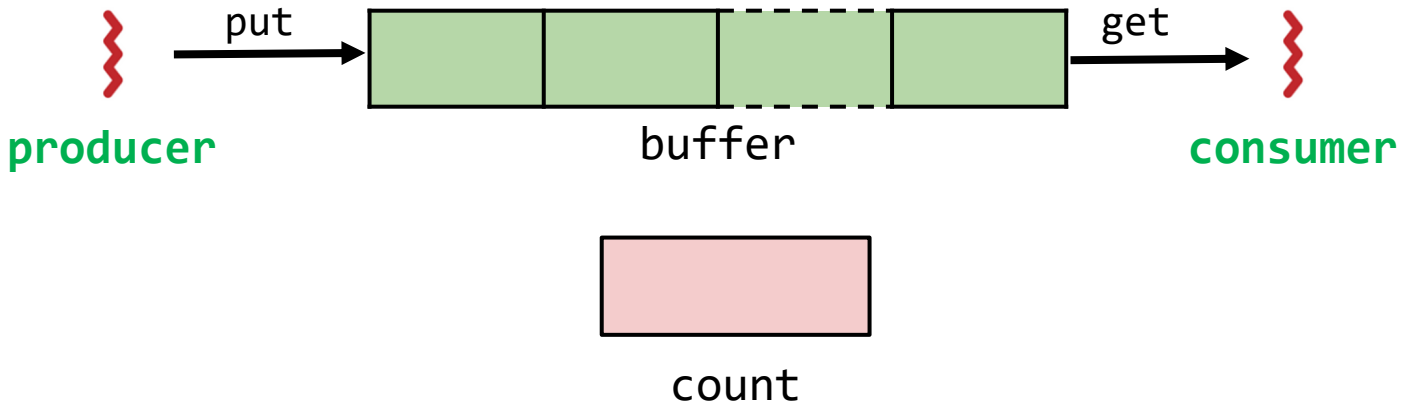
Solución: Agregar mas slots en el buffer

- **Mejora de concurrencia**: Permite que se produzca y se consuma de manera concurrente.
- **Mejora de eficiencia**: Reduce cambios de contexto.

```
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
  buffer[fill_ptr] = value;
  fill_ptr = (fill_ptr + 1)%MAX;
  count++;
}

int get() {
  int tmp = buffer[use_ptr];
  use_ptr = (use_ptr + 1)%MAX;
  count--;
  return tmp;
}
```



Productor/consumidor – Solución final

Solución: Agregar mas slots en el buffer

```
cond_t empty, fill;
mutex_t mutex;
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&mutex);          // p1
        while (count == MAX)                 // p2
          pthread_cond_wait(&empty, &mutex); // p3
        put(i);                              // p4
        pthread_cond_signal(&fill);          // p5
        pthread_mutex_unlock(&mutex);        // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
    pthread_mutex_lock(&mutex);           // c1
    while (count == 0)                    // c2
      pthread_cond_wait(&fill, &mutex);   // c3
    int tmp = get();                      // c4
    pthread_cond_signal(&empty);          // c5
    pthread_mutex_unlock(&mutex);         // c6
    printf("%d\n", tmp);
  }
}
```

- Un **productor va a dormir** solo si el buffer está **lleno** (**p2**).
- Un **consumidor va a dormir** sólo si el buffer está **vacío** (**c2**).

Productor/consumidor – Solución final

Código 1

Aspectos a resaltar:

- A diferencia de los casos anteriores, este código ya tiene en cuenta que el buffer puede tener más de un elemento.
- No maneja marca de terminación de modo que aunque funciona el código se bloquea.

Archivos:

- mythreads.h
- main_buffer-v3.c

Compilación:

```
gcc -I. main_buffer-v3.c -Wall -lpthread -o bounded-buffer.out
```

Ejecución:

```
bounded-buffer.out <buffersize> <loops> <consumers>
```

Productor/consumidor – Solución final

Código 1

Escenario	Conclusiones
Un solo productor y varios consumidores	El código funciona pero se bloquea

```
$ ./bounded-buffer.out 10 1000 4
```

```
989
990
991
992
993
994
995
996
997
998
999
981
978
979
^C
```

Productor/consumidor – Solución final

Código 2

Aspectos a resaltar:

- A diferencia de los casos anteriores, este código ya tiene en cuenta que el buffer puede tener más de un elemento.
- Corrige el problema del código anterior agregando un -1 como condición de terminación al meter datos.

Archivos:

- mythreads.h
- main_buffer-v4.c

Compilación:

```
gcc -I. main_buffer-v4.c -Wall -lpthread -o bounded-buffer.out
```

Ejecución:

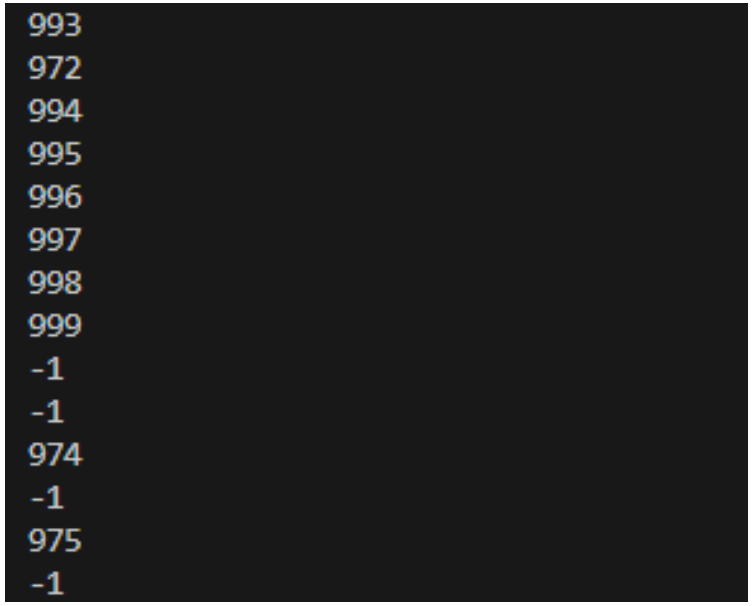
```
./bounded-buffer.out <buffersize> <loops> <consumers>
```

Productor/consumidor – Solución final

Código 2

Escenario	Conclusiones
Un solo productor y varios consumidores	El código funciona

```
$ ./bounded-buffer.out 10 1000 4
```



Resumen: Reglas de juego

- Mantenga un **estado** además de las **variables condición** (CV).
- Siempre espere/señale (**wait/signal**) el lock adquirido.
- Siempre que adquiera un lock, verifique el **estado**.



count

```
int loops; // must initialize somewhere...
cond_t empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
  int i;
  for (i = 0; i < loops; i++) {
    pthread_mutex_lock(&mutex);           // p1
    while (count == 1)                    // p2
    pthread_cond_wait(&empty, &mutex);    // p3
    put(i);                               // p4
    pthread_cond_signal(&fill);           // p5
    pthread_mutex_unlock(&mutex);         // p6
  }
}
```

```
void *consumer(void *arg) {
  int i;
  while(1) {
  pthread_mutex_lock(&mutex);           // c1
    while (count == 0)                  // c2
    pthread_cond_wait(&fill, &mutex);   // c3
    tmp = get();                        // c4
    pthread_cond_signal(&empty);        // c5
    pthread_mutex_unlock(&mutex);       // c6
    printf("%d\n", tmp);
  }
}
```

Referencias

- Diapositivas y videos oficiales del curso
- https://adit.io/posts/2013-05-15-Locks,-Actors,-And-STM-In-Pictures.html
- https://link.springer.com/chapter/10.1007/978-1-4842-4398-5_5
- https://preshing.com/20150316/semaphores-are-surprisingly-versatile/
- https://jvns.ca/teach-tech-with-cartoons/
- https://code-cartoons.com/
- https://medium.com/basecs
- https://xkcd.com/
- https://www.qwantz.com/
- https://pbfcomics.com/
- http://turnoff.us/
- http://turnoff.us/geek/arduino-project/
- https://prepinsta.com/operating-systems/critical-section-problem/
- https://github.com/capedrazab/os-20191/wiki
- https://github.com/jjpulidos/Operating-Systems-UNAL-20191
- https://github.com/capedrazab/os-20201/wiki

Referencias

- http://csl.snu.ac.kr/courses/4190.307/2020-1/
- https://brendangregg.com/linuxperf.html
- http://pages.cs.wisc.edu/~harter/537/slides.html
- http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html
- http://odl.sysworks.biz/disk$axpdocsep012/network/dcev30/develop/appdev/Appde179.htm
- https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html
- https://ocw.unican.es/course/view.php?id=240&section=1
- https://omscs.gatech.edu/cs-6200-introduction-operating-systems
- https://ruslanspivak.com/lsbaws-part1/
- https://prepinsta.com/operating-systems/threads/
- http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/

Referencias

- https://github.com/capedrazab/os-20211
- https://github.com/pkgamma/cs241-coursebook-clone-fa19/wiki/Synchronization
- https://github.com/tiemma/udacity-operating-systems
- https://github.com/monorinam/GaTech-CS-6210-Advanced-Operating-Systems
- https://github.com/stevenxchung/Introduction-to-Operating-Systems
- https://github.com/stevenxchung/Introduction-to-Operating-Systems/tree/master/Lesson%2014%20-%20Synchronization%20Constructs
- https://github.com/khanhnamle1994/operating-systems
- https://github.com/gauti1311/SystemMonitor
- https://github.com/bresilla/robond_project_2
- https://github.com/Maxxquoi/Introduction-to-Operating-Systems
- https://github.com/sergioarmgpl/operating-systems-usac-course
- https://github.com/MrinmoiHossain/Online-Courses-Learning
- https://github.com/PKUFlyingPig/MIT6.S081-2020fall
- https://github.com/mishal23/os-lab

Referencias

- https://github.com/mishal23/os-simulator
- https://github.com/zxc479773533/HUST-OS-Course-Design
- https://github.com/mauri870/baking-pi
- https://github.com/hyojeonglee/osfall2019
- https://github.com/JoyeBright/OSLab
- https://github.com/Matrixpecker/VE482-public
- https://github.com/MrinmoiHossain/Robot-Operating-System-Udemy
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/3-Threads-and-Concurrency.md
- http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html
- https://levelup.gitconnected.com/producer-consumer-problem-using-condition-variable-in-c-6c4d96efcbbc
- http://jakascorner.com/blog/2016/01/condition-variable.html
- https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html
- https://github.com/CodeExMachina/ConditionVariable

Referencias

- http://csl.snu.ac.kr/courses/4190.307/2020-1/
- https://brendangregg.com/linuxperf.html
- http://pages.cs.wisc.edu/~harter/537/slides.html
- https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html